

AGRIAI 2.0 | FORENSIC SOFTWARE EXAMINATION

AI Functions and Model Orchestration

Exact prompts, model calls, grounding, fallbacks, vision, speech and image logic

Volume 02

Prepared and authored solely by Thaddeus Nii Teiko Tagoe

Founder, designer and sole creator of AgriAI

Computer Science, University of Ghana

Repository examined: ThaddeusHackz/agriai-v2

Evidence commit: 991457272b7fec548e02c645c5593b930af5ea29

Examination date: 18 August 2026

Classification: Internal technical record

Technical basis

This document is grounded in direct source review, route enumeration, dependency audit, secret pattern scanning, strict TypeScript checking, ESLint, production compilation, PostgreSQL integration tests and local HTTP behavior tests. Live third party AI accounts were not available in the examination environment, so provider success is assessed from code paths and contracts rather than a claim of live production availability.

1. What makes the AI possible

AgriAI does not train or ship its own neural network. Intelligence comes from provider APIs plus Ghana-specific orchestration. `lib/ai.ts` creates a Gemini client, chooses models, sends prompts and content, parses provider output and supplies local answers. `app/api/chat/route.ts` adds mode, language, conversation history and optional web context. `lib/cloudflare.ts` supplies a second remote language model. `lib/db.ts` stores the editable prompt and local knowledge rows.

2. Secret and client initialization

```
firstEnv(names):
  for each environment name:
    trim whitespace and surrounding quotes
    reject empty, YOUR_*, CHANGE_ME* and placeholder* values
    return the first accepted value

geminiApiKey aliases:
  GEMINI_API_KEY
  GOOGLE_API_KEY
  GOOGLE_GENERATIVE_AI_API_KEY
  GOOGLE_GENAI_API_KEY
  GOOGLE_GEMINI_API_KEY
```

`getGemini()` caches a `GoogleGenAI` object and the key used to build it. A changed environment value causes client reconstruction. `geminiConfigured()` checks only whether a recognized value exists. It does not verify the key until a model call is attempted.

3. Chat model waterfall

Order	Configured model name	Role
0	Admin preferred settings.chat.model	Inserted first when non-empty
1	gemini-2.5-flash	Primary seeded chat and vision model
2	gemini-2.5-flash-lite	Lower-cost Gemini fallback
3	gemini-2.0-flash	Compatibility fallback
4	gemini-2.0-flash-001	Pinned compatibility fallback
5	gemini-flash-latest	Alias fallback
6	gemini-1.5-flash	Legacy fallback
7	gemini-1.5-flash-latest	Legacy alias fallback

`uniqueModels()` removes duplicate names while preserving order. `geminiGenerateText()` and `geminiGenerateStream()` catch each model error, log the model and continue. For names matching 2.5, `thinkingBudget` is set to zero. A non-empty response ends the waterfall. If all models fail, the last error is thrown to the route orchestrator.

4. Exact chat request transformation

1. Parse JSON. Invalid JSON returns HTTP 400.
2. Trim message and cap it at 2,000 characters. Empty content returns HTTP 400.
3. Whitelist language as en, tw, ga, ee, ha or fr. Default to en.
4. Whitelist mode as expert or agent. Any other value becomes standard.
5. Keep at most 10 client history entries, only user or assistant roles, with 4,000 characters each.
6. If `webSearch` is true, search for the farmer message plus the literal suffix Ghana 2026.
7. Read settings.chat and select `systemPrompt`, `expertPrompt` or `agentPrompt`.
8. Append `languageInstruction` and the literal identity You are AgriAI.
9. If sources exist, append their numbered text into the Gemini system instruction.

10. Stream Gemini. If no complete answer remains, call Cloudflare. If that fails, call localAnswer.

11. Persist user and assistant turns, then send the done event.

5. Prompt control plane

Mode	Stored field	Technical effect
standard	settings.chat.systemPrompt	Concise practical Ghana farming assistance
expert	settings.chat.expertPrompt	Requests agronomic detail, rates, densities, IPM and economics
agent	settings.chat.agentPrompt	Requests question decomposition, current evidence and structured synthesis

The prompts are mutable at runtime through `/api/admin/settings`. This is powerful because behavior changes without deployment. It is also a security boundary because any authenticated editor can currently modify chat settings. Prompt version, approval, rollback and audit history are not stored.

6. Language steering

`lib/languages.ts` maps English, Twi, Ga, Ewe, Hausa and French. The language function appends a text instruction to the model. It is not a translation engine and it does not verify that the answer actually uses the requested language. Vision is weaker: for non-English selections it requests English with a short summary in the farmer language. Local knowledge answers are generally English even when a non-English code is selected.

7. Server Sent Events protocol

```
event: delta
data: {"text":"a model text fragment"}

... repeated while Gemini streams ...

event: done
data: {
  "text":"the complete answer",
  "demo":false,
  "sources":[{"title":"...", "url":"..."}],
  "searchDemo":false,
  "provider":"gemini"
}
```

The response headers are `text/event-stream`, `no-cache`, `no-transform`, `keep-alive` and `X-Accel-Buffering:no`. `Chat.tsx` reads bytes, splits frames on blank lines, parses the data JSON and appends delta text. The final done payload replaces the temporary assistant content if it contains text. Cloudflare does not stream in this implementation, so the UI receives its complete text only in done.

8. Search grounding

`searchWeb()` uses Tavily advanced search with `include_answer` and six results when `TAVILY_API_KEY` exists. On no key or failure, `keylessSearch()` calls English Wikipedia OpenSearch and DuckDuckGo Instant Answer. `contextBlock()` converts the answer and snippets into numbered plain text. The chat route places this block inside the system instruction and tells the model to cite [1], [2]. Source title and URL metadata also travels to the browser.

Grounding limitation

This is retrieval-augmented prompting without embeddings, ranking, content sanitization or source trust scoring. Search snippets can contain adversarial instructions. A citation marker shows which snippet the model associated with a statement; it does not prove the statement is correct.

9. Local knowledge engine

localAnswer() lowercases the question and scans every database knowledge row. The first row with any keyword contained in the question wins. If no keyword hits, a 12-character question prefix check runs. Seven regular-expression fallback topics follow. The final response is a generic capability card. There is no semantic embedding, BM25 score, stemming, language-specific matching or confidence calculation.

```
localAnswer(question):
  for knowledge row in insertion order:
    if any keyword is a substring of question: return row.answer
    if question contains first 12 chars of stored question: return row.answer
  for curated regular expression answer:
    if expression matches: return answer
  return generic offline help card
```

10. Crop disease vision

1. DiseaseDetector.tsx reads a selected file into a browser data URL.
2. POST /api/vision accepts image and language. It checks string length from 100 to 4.5 million characters.
3. detectCropDisease() parses the data URL into mimeType and base64 data.
4. A prompt demands a strict JSON object with detected, confidence, description and treatment.
5. The preferred vision model comes from settings.chat.visionModel, followed by the chat model waterfall.
6. Temperature is 0.2 and max output is 1,024 tokens.
7. A broad brace regular expression extracts JSON, then the fields are normalized and confidence is clamped from 0 to 100.
8. Missing key or total failure returns FALLBACK_DISEASE with demo:true.

No specialized disease classifier, labeled crop dataset, image preprocessor or calibration model is present. The confidence is generated by Gemini, not statistically calibrated by AgriAI. The seeded 92 percent metric is not supported by a checked-in evaluation suite.

11. Image generation

POST /api/generate caps the prompt at 500 characters and requires a case-insensitive substring from SAFE_SUBJECTS. It adds a photographic suffix. Cloudflare Flux-1-schnell is attempted with four steps. Gemini image models are attempted only if Cloudflare does not return a data URL. Success returns {ok, image, provider}. The image is represented in JSON as base64, which increases payload size and memory use.

Priority	Provider or model	Output
1	Cloudflare @cf/black-forest-labs/flux-1-schnell	PNG data URL
2	gemini-2.5-flash-image	First inline image part
3	gemini-2.0-flash-preview-image-generation	First inline image part
4	gemini-2.0-flash-exp-image-generation	First inline image part

12. Voice input and output

Chat.tsx first tries the browser SpeechRecognition interface. If unavailable, MediaRecorder captures an audio blob and posts multipart FormData to /api/transcribe. That route permits up to 25 MB, base64 encodes the complete file and asks Gemini 2.5 Flash to transcribe verbatim. Output speech posts up to 1,500 characters to /api/tts. ElevenLabs eleven_multilingual_v2 returns MPEG audio. English and French use the Rachel voice identifier, while other languages use the default multilingual identifier.

13. Provider failure matrix

Capability	Primary	Fallback	Last behavior
Chat	Gemini stream	Cloudflare Llama	Local knowledge, HTTP 200 SSE, demo true
Search	Tavily	Wikipedia and DuckDuckGo	Empty sources, demo true
Vision	Gemini multimodal	Other Gemini model names	Offline disease card, demo true
Image	Cloudflare Flux	Gemini image models	HTTP 502 or 503
Transcription	Gemini	None	HTTP 501 missing key or 500 failure
TTS	ElevenLabs	None	HTTP 501 missing key or 502 failure
Weather	OpenWeatherMap	Open-Meteo	Seeded deterministic forecast

14. The technical answer to how AI works

The model is only one stage. AgriAI works because the route constructs a controlled request, attaches Ghana context, attaches language and mode instructions, optionally adds retrieved evidence, feeds recent conversation turns, handles token streaming, catches provider failure, emits a stable client protocol and records the result. The application engineering around the model creates the product experience.

Authorship and control statement

This volume was prepared, authored and issued solely under the name of Thaddeus Nii Teiko Tagoe. This volume is the authoritative technical explanation of every AI-enabled function visible in the examined source snapshot. The repository records identify Thaddeus Tagoe as the founder, sole designer, developer and maintainer of AgriAI.